Emotion Detection from Social Media Posts

with Ambiguity Detection

A report submitted for the course named Project I (CS3201)

Submitted By

ARJEET SINGH

SEMESTER VI

230101078

Supervised By

DR. RAJKUMARI BIDYALAKSHMI DEVI

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY SENAPATI, MANIPUR

APRIL 2026

Declaration

In this submission, I have expressed my idea in my own words, and I have adequately cited and referenced any ideas or words that were taken from another source. I also declare that I adhere to all principles of academic honesty and integrity and that I have not misrepresented or falsified any ideas, data, facts, or sources in this submission. If any violation of the above is made, I understand that the institute may take disciplinary action. Such a violation may also engender disciplinary action from the sources which were not properly cited or permission not taken when needed.

Arjeet Singh
230101078

DATE:

Department of Computer Science and Engineering
Indian Institute of Information Technology Senapati, Manipur

Rajkumari Bidyalakshmi Devi
Assistant Professor, CSE, IIIT Manipur

Email: [REDACTED]
Contact No: [REDACTED]

To Whom It May Concern

This is to certify that the project report entitled "**Emotion Detection from Social Media Posts with Ambiguity Detection**," submitted to the department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur in partial fulfilment for the award of degree of Bachelor of Technology in Computer Science and Engineering is a record of bonafide work carried out by **Arjeet Singh** bearing roll number **230101078** for the course Project I (CS3201).

Signature of the Supervisor

(Rajkumari Bidyalakshmi Devi)

Signature of the Examiner 1 .....

Signature of the Examiner 2 .....

Signature of the Examiner 3 .....

Signature of the Examiner 4 .....

Abstract

This work presents a novel approach for mental health and emotion detection from social media text, addressing the limitations of conventional Natural Language Processing (NLP) models in handling ambiguity and contextual uncertainty. Traditional models based on TF-IDF and classification algorithms often fail to capture nuanced emotional states, particularly in sentences with overlapping or unclear sentiment.

To overcome this, the proposed system integrates Topological Data Analysis (TDA) with machine learning to analyze the geometric structure of text embeddings. By applying persistent homology on vector representations, the model identifies structural patterns that indicate ambiguity in emotional expression. The classification pipeline is built using TF-IDF features combined with a Logistic Regression model, ensuring computational efficiency while maintaining interpretability.

A key contribution of this work is the introduction of an ambiguity detection mechanism, enabling the system to flag uncertain emotional predictions rather than forcing incorrect classifications. Experimental results demonstrate that while the baseline model achieves moderate performance, the integration of TDA enhances the system's ability to identify ambiguous inputs and improves overall robustness.

This approach provides a foundation for more reliable mental health monitoring systems, where understanding uncertainty is as critical as prediction accuracy.

Acknowledgement

I am grateful to my project supervisor, Dr. Rajkumari Bidyalakshmi Devi, for her constant guidance and insightful direction in the development of this project. Her expertise and constructive feedback throughout the development process were instrumental in shaping both the architecture and the evaluation methodology of this project. Her encouragement motivated me to tackle the challenging aspects of the system, particularly the topological data analysis and the ambiguity detection framework.

I would also like to thank my peers at IIIT Manipur for their valuable discussions and feedback during the development and testing phases of this project.

Arjeet Singh

Contents

Declaration **i**

Certificate **ii**

Abstract **iii**

Acknowledgement **iv**

1 Introduction **1**
1.1 Introduction 1
1.2 Background and Motivation 1
1.3 Problem Statement 2
1.4 Objectives 2
1.5 Scope 3
1.6 Significance and Contribution 3
1.7 Report Structure 3

2 Literature Survey **5**
2.1 Overview 5
2.2 Sentiment Analysis 5
2.3 Emotion Detection and the GoEmotions Dataset 6
2.4 Ambiguity and Uncertainty in Natural Language Processing 7
2.5 Topological Data Analysis in Natural Language Processing 7
2.6 Sentence Embeddings 8
2.7 Gaps in Existing Work and How This Project Addresses Them 8

3 Requirement Engineering **10**
3.1 Overview 10
3.2 Stakeholder Identification 10
3.3 Functional Requirements 10
3.4 Non-Functional Requirements 11
3.5 Data Requirements 12
3.6 Class Imbalance Handling Requirements 12
3.7 Ambiguity Detection Rules 12
3.8 Technology Stack 13
3.9 Requirements Summary 13

4 System Design **15**
4.1 Overview 15
4.2 System Architecture Overview 15

4.3 Feature Extraction Design . . . 16
4.3.1 Semantic Embedding . . . 17
4.3.2 TF-IDF Feature . . . 17
4.3.3 Topological Features . . . 17
4.4 Design Justification . . . 18
4.4.1 Why Topology Instead of Only Confidence . . . 18
4.4.2 Why 24 Nearest Neighbours . . . 18
4.5 Classification Design . . . 19
4.6 Ambiguity Detection Design . . . 19
4.7 Data Flow Modeling . . . 20
4.7.1 Level 0: Context Diagram . . . 20
4.7.2 Level 1: Detailed Data Flow . . . 21
4.7.3 Level 2: Ambiguity Analysis Sub-Flow . . . 22
4.8 Training vs Inference . . . 22

5 Implementation **23**
5.1 Overview . . . 23
5.2 Development Environment . . . 23
5.3 Dataset Preparation . . . 24
5.4 Text Processing Strategy . . . 24
5.5 Handling Class Imbalance . . . 25
5.6 Training Pipeline . . . 26
5.7 Inference and Ambiguity Detection . . . 26
5.8 Example Outputs . . . 27
5.9 Visualization . . . 28
5.10 Model Storage . . . 31

6 Results and Evaluation **33**
6.1 Overview . . . 33
6.2 Experimental Setup . . . 33
6.3 Classification Performance . . . 33
6.4 Understanding the Results . . . 34
6.5 Why the Accuracy Frames the Contribution Correctly . . . 34
6.6 Ambiguity Detection Results . . . 35
6.7 A Known Limitation: I Am Fine . . . 36
6.8 System Testing . . . 36
6.8.1 Black Box Testing . . . 36
6.8.2 Clear Positive Inputs . . . 36
6.8.3 Clear Negative Inputs . . . 37
6.8.4 Neutral Inputs . . . 37
6.8.5 Ambiguous Inputs . . . 37
6.8.6 Edge Cases . . . 37
6.8.7 White Box Testing . . . 38
6.8.8 Case Study 1: I Am Fine . . . 38
6.8.9 Case Study 2: I Do Not Know How I Feel . . . 40
6.9 Timeline of Project Development . . . 41
6.10 Topology Analysis . . . 41
6.11 Threshold Sensitivity . . . 42

6.12 Performance 42
6.13 Limitations 42

7 Conclusion and Future Work **44**
7.1 Summary 44
7.2 What Was Achieved 44
7.3 Advantages of the Approach 45
7.4 Limitations 45
7.5 Future Work 45
7.6 Final Thoughts 46

Bibliography **47**

List of Figures

4.1 System Architecture for Emotion Detection with Ambiguity Detection 16
4.2 Feature Extraction Pipeline 18
4.3 Ambiguity Detection Activity Flowchart 20
4.4 Level 0 Context Diagram 21
4.5 Level 1 Data Flow Diagram (DFD) 21
4.6 Level 2 DFD: Process 3.0 (Ambiguity Analysis) 22

5.1 Point Cloud Visualization of 24 Nearest Neighbours (PCA Projection) 29
5.2 Persistence Diagram Showing H0 and H1 Topological Features 30
5.3 Persistence Landscape Representation 31

6.1 Visualization for Case Study 1: “I Am Fine” — Clear Topology 39
6.2 Visualization for Case Study 2: “I Dont Know How I Feel” — Complex Topology 40

List of Tables

2.1 Comparison of Existing Methods on the GoEmotions Dataset 6

3.1 Technology Stack 13
3.2 Summary of Key System Requirements 14

5.1 GoEmotions Label Mapping to Three Classes 24
5.2 Text Processing Strategy Comparison 25
5.3 Example System Inputs and Outputs 28

6.1 Classification Performance on GoEmotions Test Set 34
6.2 Comparison of Proposed Model with Existing GoEmotions-Based Approaches . 35
6.3 Example Inputs and Ambiguity Detection Results 35
6.4 Black Box Test Cases for Ambiguity Detection 37
6.5 Project Development Timeline 41
6.6 Effect of Loop Threshold on Ambiguity Rate 42

Chapter 1

Introduction

1.1 Introduction

Social media has changed the way people communicate. Platforms like Twitter, Reddit, and Instagram have become spaces where people share their thoughts, feelings, and reactions in real time. Every day, millions of posts are written. Some express clear happiness or sadness. Many others are confusing, sarcastic, or somewhere in between.

Understanding the emotion behind a social media post is useful for many reasons. Companies want to know what customers think about their products. Researchers study mental health trends by analyzing what people write online. News organizations want to understand how the public is reacting to events. In all these cases, someone or something needs to read large amounts of text and decide what emotion is being expressed.

This project builds a system that does exactly that. It reads a social media post and decides whether the emotion is Positive, Negative, Neutral, or Ambiguous. The last category is what makes this project different from most existing systems.

1.2 Background and Motivation

Emotion detection from text is not a new problem. Researchers have been working on it for many years. But most systems that have been built share one common limitation: they always give a definite answer. They look at the input, run some computation, and return one class. They do this even when the text is genuinely unclear.

Consider the sentence "I am fine." On the surface, it looks positive. But in many situations, it is used sarcastically. A person who is upset but does not want to talk about it might say "I am fine" and mean exactly the opposite. A standard classifier will almost always return POSITIVE for this sentence. That is not really honest.

This project is built on the idea that uncertainty is a valid and useful output. When a system does not know something, it should say so. This is especially important for emotion detection, where the cost of a wrong confident prediction can be higher than simply flagging the input as unclear.

The second motivation for this project is the use of geometric structure to detect ambiguity. When text is converted into a vector using a language model, texts with similar meanings end up close together in that vector space. What this project asks is: what does the neighbourhood

around an input look like geometrically? If the neighbourhood is clean and well-organized, the input is probably easy to classify. If the neighbourhood is complicated, the input is probably in a border region between emotion clusters, which is a sign of ambiguity.

To measure this geometric complexity, the project uses Topological Data Analysis (TDA), specifically a technique called persistent homology. This is not something that has been tried for ambiguity detection in social media emotion analysis before, which is the research contribution this project makes. Recent studies on emotion topology [14] have explored the fundamental components of emotions from text using word embeddings, but the application of local TDA for ambiguity detection remains a novel frontier.

A third motivation comes from looking at how previous models have performed on the GoEmotions dataset. Models like BERT-based classifiers and SVM systems that were trained on GoEmotions all struggle with the same problem: they do very well on the majority classes and poorly on minority ones, and they never say when they are unsure. The GoEmotions dataset is also known to have a lot of label noise and ambiguous samples because it was labelled by crowd workers who did not always agree. Furthermore, standard models often miss nuances such as sarcasm [20], leading to misleading results. Standard models treat all samples as equally clear, which is a design flaw this project tries to address.

1.3 Problem Statement

The problem addressed in this project can be stated as follows. Given a short piece of text from a social media platform, the objective is to build a system that:

1. Classifies the text into one of three emotion categories: Positive, Negative, or Neutral.
2. Determines whether the text is too ambiguous to be classified confidently, and if so, returns an Ambiguous label instead.

The challenge is not merely building an accurate classifier, but determining when a classification should not be made. Standard machine learning models are designed to optimize accuracy over a fixed set of classes and therefore always produce a definitive prediction, even when the input is inherently unclear.

Such models lack a mechanism to represent uncertainty or ambiguity in human language. This project addresses that limitation by introducing a system that is capable of identifying and explicitly handling ambiguous emotional expressions, rather than forcing potentially misleading classifications.

1.4 Objectives

The main objectives of this project are as follows.

1. Build a sentiment classification system that works on informal social media text from the GoEmotions dataset.
2. Use Topological Data Analysis, specifically Vietoris-Rips persistent homology, to measure the geometric complexity of the neighbourhood of each input in the embedding space.

3. Combine topological complexity with classifier confidence to produce a principled ambiguity signal.
4. Map the 28-class GoEmotions labels to three classes (Positive, Negative, Neutral) in a way that preserves the semantic groupings.
5. Handle the class imbalance in the dataset without using synthetic data generation techniques that would distort the topological structure.
6. Deploy the final system as an interactive command-line tool that explains its decisions.

1.5 Scope

The system works with English text. It is trained on Reddit comments from the GoEmotions dataset. It returns one of four outputs: Positive, Negative, Neutral, or Ambiguous. The system is designed to run on a standard laptop CPU for inference, though training benefits from GPU availability.

The system does not handle audio, images, or video. It does not track user history or learn from new inputs after training. It operates as a single-query system where each input is classified independently.

1.6 Significance and Contribution

The key contribution of this project is the use of **persistent homology as an ambiguity signal at inference time**. When a new text arrives, the system finds its nearest neighbours in the training embedding space and computes a fresh Vietoris-Rips persistence diagram from those neighbours. The number of one-dimensional holes (H1 features) in that diagram measures how geometrically tangled the neighbourhood is. A high count means the input is in a complicated part of the space, which the system interprets as a sign of ambiguity.

This is different from existing approaches to uncertainty in NLP in two ways. First, it is structural rather than just probabilistic. Most uncertainty methods rely on dropout-based sampling or ensemble disagreement. This project uses the actual geometry of the data. Second, it is computed fresh at inference time using real training data, not a precomputed approximation.

The approach is also interpretable. When the system flags a text as Ambiguous, it can explain exactly which rule triggered the decision: extreme loop count, moderate loop count combined with low confidence gap, or very low confidence gap alone. This kind of explanation is useful in practice.

1.7 Report Structure

The remainder of this report is organized into seven chapters, each addressing a specific aspect of the project's development and evaluation:

- **Chapter 2: Literature Survey** reviews the foundational research in sentiment analysis, emotion detection, handling uncertainty in NLP, and the application of Topological Data Analysis to text.

- **Chapter 3: Requirements Analysis** outlines the functional and non-functional requirements of the proposed system, including data specifications and design constraints.
- **Chapter 4: System Design and Architecture** details the high-level architecture, component interactions, and the data flow through the emotion detection pipeline.
- **Chapter 5: Implementation Details** provides an in-depth explanation of the technical implementation, including embedding strategies and topological feature extraction.
- **Chapter 6: Results and Evaluation** presents the experimental results, performance metrics on the GoEmotions dataset, and qualitative case studies.
- **Chapter 7: Conclusion and Future Work** summarizes the findings of the project, discusses the identified limitations, and outlines potential directions for future research.

Chapter 2

Literature Survey

2.1 Overview

This chapter reviews the existing work that is relevant to this project. The main areas are sentiment analysis and emotion detection, handling ambiguity and uncertainty in natural language processing, topological data analysis for text, and the specific problems that have been observed with existing models on the GoEmotions dataset.

2.2 Sentiment Analysis

Sentiment analysis is the task of identifying whether a piece of text expresses a positive, negative, or neutral opinion. It is one of the oldest and most studied problems in natural language processing.

to individual words and then combining those scores to get an overall sentiment. This is a well-known approach for early systems. These were designed specifically for social media text and handled punctuation, capitalization, and emoticons as signals of sentiment intensity. Such systems are fast and work reasonably well for clear, unambiguous text. However, they often fail on sarcasm and do not have any mechanism to say when it is unsure.

Machine learning approaches improved on this. Systems based on Naive Bayes, Support Vector Machines (SVM), and logistic regression used bag-of-words representations. These models could learn from examples and generalize beyond a fixed vocabulary. They performed better on benchmark datasets but still struggled with informal language, negations, and short posts where context is limited.

The deep learning era brought significant improvements. Previous models applied convolutional filters over word embeddings to capture local phrases and patterns. Recurrent networks, especially LSTMs, captured longer dependencies in a sequence. Attention mechanisms allowed models to focus on the most relevant parts of the input.

BERT [2] changed the field significantly. It was pre-trained on a massive amount of text using a masked language modeling objective, which gave it a deep understanding of context. Fine-tuned BERT models reached state-of-the-art results on almost every sentiment and emotion benchmark. RoBERTa and DeBERTa further refined this approach.

The problem is that all of these systems, from VADER to BERT fine-tuned models, **always return a label**. They have no built-in way to say that the input is unclear. They are trained to

minimize classification error on a fixed label set, and that is all they do.

2.3 Emotion Detection and the GoEmotions Dataset

Emotion detection is a more specific version of sentiment analysis. The goal is to identify which particular emotion a text expresses, such as joy, sadness, anger, fear, or disgust. This is harder than binary sentiment classification because the boundaries between emotions are not sharp.

The GoEmotions dataset [1] is one of the most important resources for this task. It was released by Google and contains around 58,000 Reddit comments labelled by crowd workers with 28 emotion categories. Each comment can have multiple labels. The dataset is publicly available and has been widely used in research.

Several important problems have been identified with models trained on GoEmotions:

The first is **class imbalance**. The Neutral class makes up a large portion of the dataset. Many specific emotion categories like grief, nervousness, and pride have very few examples. Models trained on this data tend to predict the majority classes much more often and perform poorly on minority classes. This is visible in the classification reports of most published systems on this dataset.

The second problem is **label ambiguity**. Because the dataset was labelled by crowd workers, there is significant disagreement between annotators. A comment like "Well, that happened" received different labels from different people. Some saw it as neutral, some as surprised, some as slightly negative. The final label is determined by majority vote, which hides this uncertainty. Models trained on such data are essentially learning to predict the most common human interpretation, not the correct one, and there may not be a single correct one.

The third problem is the presence of **informal language**. Reddit comments contain slang, abbreviations, and expressions that are not well-represented in standard training data. Words like "yolo", "fomo", "lol", and "idk" carry emotional meaning but are often handled poorly by models trained on more formal text. The embedding models used to represent these words may not give them accurate positions in the semantic space.

Several BERT-based systems have been published on GoEmotions [8, 17]. Most report accuracy in the range of 0.46 to 0.58 on the full 28-class problem [13, 12]. On the simplified 3-class version (Positive, Negative, Neutral), results are higher but still reflect the difficulty of the dataset [1]. Several studies have explored emotion detection using the GoEmotions dataset. These approaches primarily rely on transformer-based architectures and deep learning models to achieve strong classification performance.

Table 2.1: Comparison of Existing Methods on the GoEmotions Dataset

Study	**Method**	**Metric**	**Score**
Demszky et al. (2020)	BERT (baseline)	Macro-F1	0.46
Sentence-BERT (2019)	Transformer Embedding	Accuracy	∼82%
BiLSTM Model (2021)	Deep Learning	Macro-F1	∼0.45
Transformer Model (2021)	Fine-tuned Transformer	Micro-F1	∼0.72
LLM-based Model (2023)	Transformer + Augmentation	Accuracy	∼85%
ModernBERT (2024)	Advanced Transformer	Accuracy	∼86–87%

From **Table 2.1**, it can be observed that transformer-based models achieve high performance on the GoEmotions dataset. However, these approaches are primarily designed for classification and do not explicitly account for ambiguity in emotional expressions.

2.4 Ambiguity and Uncertainty in Natural Language Processing

The question of how to handle uncertainty in NLP predictions has been studied in several different ways.

Bayesian deep learning approaches model uncertainty by treating model weights as distributions rather than fixed values. At test time, dropout is kept active and multiple predictions are made. The variance across these predictions estimates model uncertainty. This works but requires multiple forward passes and adds computational cost.

Conformal prediction is another framework that produces prediction sets rather than single labels. Instead of saying "this is Positive", a conformal predictor might say "this is Positive or Ambiguous, and I am 90% confident the true label is in this set." This is more honest but harder to explain to a non-technical user.

Selective prediction allows a model to abstain from making a prediction when it is below a confidence threshold. This is close to what this project does, but it relies only on the classifier's own confidence. It does not use any external signal about the structure of the data.

The key limitation of all these approaches is that they use the model's own output to estimate uncertainty. If the model is confidently wrong, none of these methods will catch it. Recent research on topological quantification of ambiguity [5, 9] suggests that geometric signals can provide a more robust measure of uncertainty. Other works such as MARCH [26] have evaluated the intersection of ambiguity interpretation and multi-hop inference, further highlighting the complexity of handling unclear inputs. What is needed is an external signal that looks at the data itself, not just at what the model says.

2.5 Topological Data Analysis in Natural Language Processing

Topological Data Analysis (TDA) is a field of mathematics that studies the shape of data. The main tool is persistent homology [3], which tracks the birth and death of topological features as a parameter is varied. The key features are connected components (H0 features) and loops or holes (H1 features).

The Vietoris-Rips complex is the most common construction used in practice. Given a set of points and a scale parameter, it builds a simplicial complex by connecting points that are closer than the scale. As the scale increases, new components connect and new loops appear and then fill in. Persistent homology records which features appear and how long they persist. Features that persist for a long time are considered genuinely structural, while features that appear and disappear quickly are considered noise.

TDA has been applied to NLP in several ways. Zhu [31] applied persistent homology to text classification and showed that topological features captured information that was not present

in bag-of-words representations. Michel et al. [29] analyzed the topology of sentence embedding spaces and found that the topology changes depending on the semantic content of the text. Recent reviews [10] have highlighted the potential of topological deep learning beyond standard persistence. In affective computing, AffectiveTDA [7] has been used to improve both analysis and explainability. Beyond NLP, TDA has found success in analyzing complex signals in fields such as cardiology [19] and neurophysiology, where it is used as a tool for EEG processing [18]. Tools such as giotto-tda [4], teaspoon [22], and TDAvec [11] have made these complex mathematical approaches more accessible to the machine learning community.

What this project does differently is apply persistent homology at the **neighbourhood level at inference time**. Instead of computing topology over the whole dataset, it computes a fresh persistence diagram for the 24 nearest neighbours of each new input. This local topology captures whether the input is sitting in a clean region of the space or in a boundary region between clusters. The number of H1 loops in this local diagram is used as a signal of ambiguity.

This approach has not been used for ambiguity detection in social media text, to the best knowledge of the author. It is the main technical contribution of this project.

2.6 Sentence Embeddings

The system uses SentenceTransformers [30] to convert text into dense vector representations. The specific model used is `all-mpnet-base-v2`, which produces 768-dimensional vectors. This model was trained to place semantically similar sentences close to each other in the vector space.

Modern embedding models like this one are trained on very large datasets and learn to handle most of the variation in natural language automatically. They do not require explicit preprocessing steps like tokenization, stopword removal, or stemming to be performed manually. In fact, applying such preprocessing can hurt performance because it removes information that the embedding model would have used.

This is an important design choice in this project and is discussed in more detail in the implementation chapter.

2.7 Gaps in Existing Work and How This Project Addresses Them

Despite the strong performance of transformer-based models on the GoEmotions dataset, existing approaches lack the ability to explicitly detect ambiguity in emotional expression. Most models are designed to assign a fixed label, even when the input contains mixed, unclear, or conflicting emotional signals.

This limitation is significant because real-world social media text often contains subtle, implicit, or ambiguous emotions that cannot be accurately represented by a single label.

Furthermore, while embedding-based methods capture semantic similarity, they do not analyze the structural complexity of the data distribution in the embedding space.

This project addresses this gap by integrating semantic embeddings with topological features to detect structurally complex regions in the embedding space. Instead of forcing a classification,

the system identifies ambiguous inputs and assigns an “Ambiguous” label when necessary.

Chapter 3

Requirement Engineering

3.1 Overview

This chapter describes what the system needs to do and how it should behave. Requirements are divided into functional requirements, which describe what the system does, and non-functional requirements, which describe how well it does it. We also describe the data requirements and the specific rules for ambiguity detection.

3.2 Stakeholder Identification

The people who would use or be affected by this system include the following.

Researchers and data analysts who want to understand emotional trends in large amounts of social media data. They need a system that is reliable and gives useful outputs, including when the system is unsure.

Developers who want to integrate emotion detection into a larger application, such as a content moderation system or a mental health monitoring tool. They need a system that is well-documented and produces structured output.

End users who interact with the system directly by typing a sentence and wanting to know its emotional content. They need clear, understandable outputs.

3.3 Functional Requirements

The following functional requirements define what the system must do.

FR1. The system must accept any piece of English text as input through a command-line interface.

FR2. The system must classify the input into one of four categories: Positive, Negative, Neutral, or Ambiguous.

FR3. The system must convert input text into dense semantic vectors using a pre-trained sentence embedding model.

FR4. The system must extract lexical features from input text using TF-IDF with a vocabulary of at least 3000 terms.

FR5. The system must find the nearest neighbours of the input in the training embedding space using cosine distance.

FR6. The system must compute Vietoris-Rips persistent homology on the neighbourhood of the input, capturing H0 and H1 features.

FR7. The system must extract a Persistence Landscape representation from the persistence diagram.

FR8. The system must combine semantic, lexical, and topological features into a single feature vector for classification.

FR9. The system must use XGBoost classifiers trained in a one-vs-rest fashion for the three emotion classes.

FR10. The system must compute the confidence gap between the top two class probabilities.

FR11. The system must apply the ambiguity decision rules to determine the final output.

FR12. The system must display debug information with each prediction, including the probabilities, confidence gap, loop count, and the trigger reason if the output is Ambiguous.

FR13. The system must load a pre-trained model from disk at startup without requiring retraining.

FR14. The system must run in an interactive loop, allowing the user to test multiple inputs in a single session.

3.4 Non-Functional Requirements

The following non-functional requirements describe the quality of the system.

NFR1. **Performance:** The system must return a prediction in under 5 seconds per input on a standard laptop CPU.

NFR2. **Portability:** The system must run on any machine with Python 3.10 or later installed. It must not require a GPU for inference.

NFR3. **Reliability:** The system must handle unusual inputs, very short inputs, and empty inputs without crashing.

NFR4. **Explainability:** When a text is flagged as Ambiguous, the system must tell the user which rule triggered the decision.

NFR5. **Reproducibility:** Given the same input, the system must always produce the same output.

NFR6. **Scalability:** The model must be trained once and saved. It must not require retraining to handle new inputs.

3.5 Data Requirements

DR1. The system must use the GoEmotions dataset (simplified version) for training and evaluation.

DR2. The 28 GoEmotions emotion categories must be mapped to three classes: Positive, Negative, and Neutral.

DR3. The full training split of approximately 43,000 samples must be used.

DR4. The full test split of approximately 5,400 samples must be used for evaluation.

DR5. The trained model, the TF-IDF vectorizer, the nearest-neighbour index, and the training embeddings must all be saved to disk after training so that inference does not require retraining.

3.6 Class Imbalance Handling Requirements

The GoEmotions dataset is not evenly distributed across the three classes. After mapping, the Neutral class has significantly more samples than Positive or Negative. The system must handle this imbalance. The following requirements apply.

IR1. The system must not use SMOTE or any other synthetic data generation technique. The reason is that SMOTE creates interpolated points in the embedding space. Since the topological analysis depends on the real geometric structure of the data, synthetic points would create artificial topology that does not correspond to real language. This would make the ambiguity detection unreliable.

IR2. The system must handle imbalance through threshold tuning. Instead of using a fixed 0.5 decision threshold for each classifier, the threshold for each class is adjusted based on the class distribution. This allows the system to be more or less sensitive to each class without distorting the underlying data.

3.7 Ambiguity Detection Rules

The ambiguity detection mechanism is a central requirement of this project. The following rules define when a text is classified as Ambiguous.

AR1. If the number of H1 loops in the local persistence diagram is 14 or more, the text is Ambiguous regardless of the classifier probabilities. This is called the *extreme geometry condition*.

AR2. If the number of H1 loops is 10 or more, and the confidence gap between the top two predicted classes is less than 0.30, the text is Ambiguous. This is called the *moderate geometry combined with low confidence condition*.

AR3. If the confidence gap between the top two predicted classes is less than 0.10, regardless of the loop count, the text is Ambiguous. This is called the *very low confidence condition*.

AR4. If none of the above conditions are met, the text receives the label of the class with the highest probability.

The thresholds (14, 10, and 0.30) were selected based on experimentation. The loop count thresholds were chosen by looking at the distribution of loop counts across samples and identifying values that separated clearly ambiguous inputs from clearly unambiguous ones. The confidence gap threshold of 0.30 was chosen because below this value, the classifier output is not stable enough to be trusted.

3.8 Technology Stack

Table 3.1 lists the key technologies used to build the system.

Table 3.1: Technology Stack

Category	Technology	Purpose
Programming Language	Python 3.10+	Core implementation
Machine Learning	XGBoost	One-vs-Rest classification
NLP Embeddings	SentenceTransformers	Semantic embeddings (all-mpnet-base-v2)
Feature Extraction	scikit-learn	TF-IDF vectorization, KNN search
Topological Analysis	Ripser	Vietoris-Rips persistent homology
Topology Processing	Persim	Persistence Landscape computation
Data Handling	NumPy, Pandas	Array operations, data manipulation
Dataset	GoEmotions	Training and evaluation data
Model Persistence	Joblib	Saving and loading trained models
Development IDE	VS Code	Code editing and debugging

The system is built entirely in Python and does not require a GPU for inference. All dependencies are installable via `pip` and are compatible with standard laptops and desktops running Windows, macOS, or Linux.

3.9 Requirements Summary

Table 3.2 summarizes the key requirements of the system.

Table 3.2: Summary of Key System Requirements

ID	Type	Requirement	Priority
FR1	Functional	Accept English text input	High
FR2	Functional	Output Positive / Negative / Neutral / Ambiguous	High
FR3	Functional	Compute local topology at inference	High
FR4	Functional	Apply ambiguity decision rules	High
FR5	Functional	Display debug information per query	Medium
FR13	Functional	Load pre-trained model from disk	High
NFR1	Non-Functional	Respond in under 5 seconds	High
NFR2	Non-Functional	Run without GPU for inference	High
NFR4	Non-Functional	Explain why text is flagged as Ambiguous	Medium
IR1	Imbalance	Do not use SMOTE or synthetic data generation	High
IR2	Imbalance	Use threshold tuning for imbalance handling	High
AR1	Ambiguity	Flag on extreme loop count (≥ 14)	High
AR2	Ambiguity	Flag on moderate topology + low confidence	High
AR3	Ambiguity	Flag on very low confidence gap (< 0.10)	High

As shown in **Table 3.2**, high priority is given to the core classification and ambiguity detection mechanisms, while user interface and debug features are treated as medium priority.

Chapter 4

System Design

4.1 Overview

This chapter describes how the system is structured. It covers the overall architecture, the feature extraction pipeline, the classification process, and the ambiguity detection mechanism. It also explains the design decisions and the reasoning behind them.

4.2 System Architecture Overview

The system operates in two main phases: the training phase and the inference phase. The training phase is performed once to build and save the model, while the inference phase is executed for each new input provided by the user.

Figure 4.1 illustrates the overall system architecture.

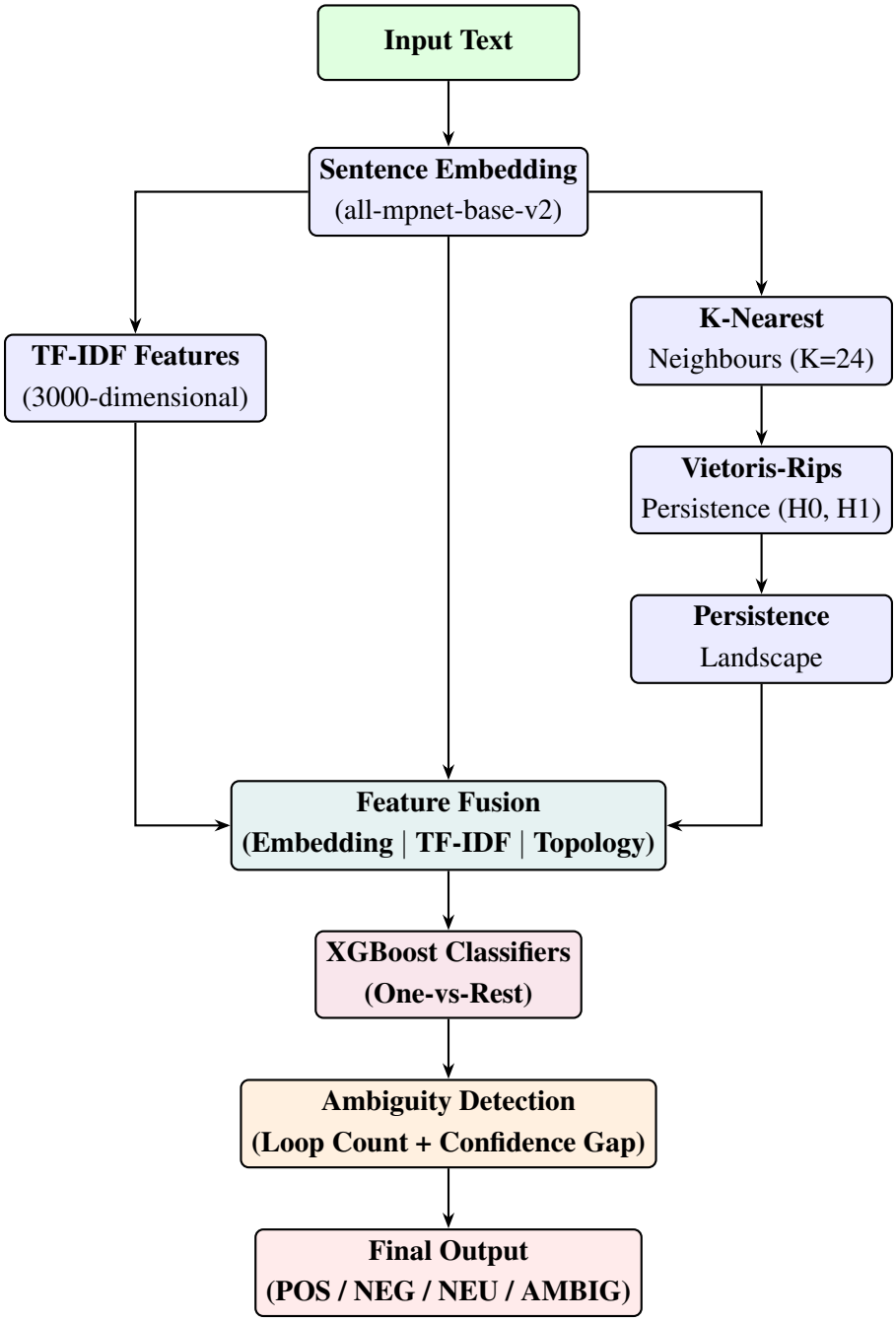

Figure 4.1: System Architecture for Emotion Detection with Ambiguity Detection

As shown in **Figure 4.1**, the system processes the input text through three distinct parallel tracks after an initial embedding step. The dense embeddings, TF-IDF representations, and topological persistence landscapes are fused into a single comprehensive feature vector. This vector is then passed to the XGBoost classifiers and finally evaluated by the ambiguity detection layer to yield the final predicted label.

4.3 Feature Extraction Design

Three types of features are extracted for each input. Each type captures a different aspect of the text.

4.3.1 Semantic Embedding

Text inputs are converted into dense vector representations using a pretrained sentence embedding model. These embeddings capture semantic meaning in a high-dimensional space.

To quantify similarity between embeddings, we use the Euclidean distance:

$$d(x_i, x_j) = \|x_i - x_j\|_2 \tag{4.1}$$

This distance metric is later used to construct topological structures over the embedding space.

4.3.2 TF-IDF Feature

In addition to semantic embeddings, textual features are extracted using the Term Frequency–Inverse Document Frequency (TF-IDF) representation.

$$w(t,d) = tf(t,d) \cdot \log\left(\frac{N}{df(t)}\right) \tag{4.2}$$

where $tf(t,d)$ is the term frequency of term t in document d, $df(t)$ is the document frequency, and N is the total number of documents.

This representation captures the importance of words within the corpus.

4.3.3 Topological Features

To capture the structural complexity of the embedding space, we employ techniques from Topological Data Analysis (TDA). This approach analyzes the shape of data by studying how points connect across multiple scales.

Vietoris–Rips Complex. A simplicial complex is constructed by connecting points whose pairwise distance is below a threshold ε:

$$\text{Edge exists if } d(x_i, x_j) \leq \varepsilon \tag{4.3}$$

Persistent Homology. Topological features such as connected components and loops are tracked across different scales. Each feature is characterized by its birth and death:

$$\text{Persistence} = d_{\text{death}} - d_{\text{birth}} \tag{4.4}$$

Features with high persistence represent significant structure, while those with low persistence are considered noise.

Loop Features (H_1). In this work, we focus on one-dimensional homology (H_1), which corresponds to loops in the data:

$$H_1 = \text{Number of 1-dimensional holes (loops)} \tag{4.5}$$

A higher number of loops indicates greater structural complexity in the embedding space.

Persistence Landscape. To transform topological information into a vector representation, persistence diagrams are converted into persistence landscapes:

$$\lambda_k(t) = k\text{-th largest value of } \{\min(t - b_i,\, d_i - t)\}_+ \tag{4.6}$$

where (b_i, d_i) denote the birth and death of the i-th topological feature, and $(\cdot)_+$ denotes the positive part.

These topological features are later combined with embedding and TF-IDF features for classification and ambiguity detection.

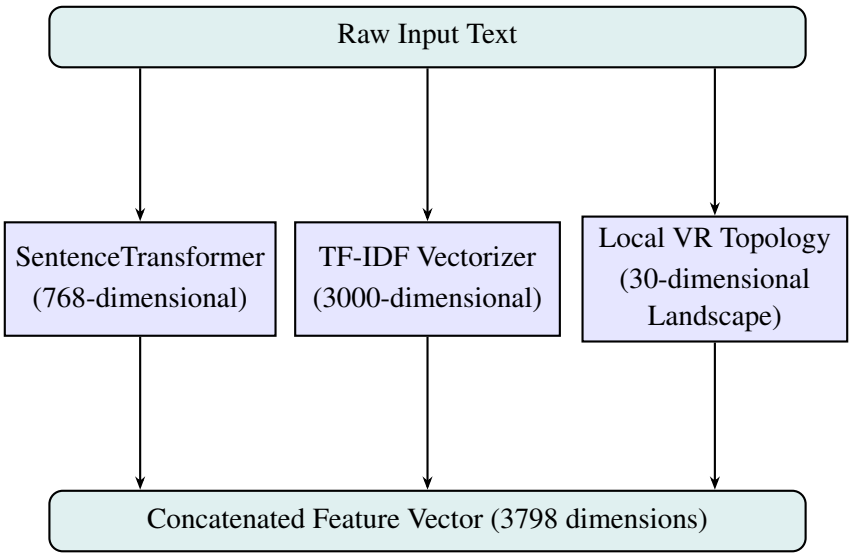

Figure 4.2: Feature Extraction Pipeline

Figure 4.2 provides a detailed view of the feature extraction pipeline. The raw text is simultaneously processed by the embedding model to capture semantic meaning, the TF-IDF vectorizer to capture keyword occurrences, and the topological data analysis module to capture geometric complexity. These three distinct feature sets are concatenated into a unified 3798-dimensional vector utilized by the downstream classifier.

4.4 Design Justification

4.4.1 Why Topology Instead of Only Confidence

Classifier confidence alone is not reliable for detecting ambiguity. A model may produce high confidence even for unclear inputs.

Topological features provide a geometric perspective. If neighbouring points belong to different emotion clusters, the structure becomes complex (more loops), indicating ambiguity.

4.4.2 Why 24 Nearest Neighbours

The number of neighbours was chosen empirically:

- Too few neighbours $\rightarrow$ insufficient structure

- Too many neighbours $\rightarrow$ loss of locality

A value of 24 provides stable and meaningful topological representations.

Since TDA depends on the real geometric structure, adding synthetic points would distort topological features and reduce reliability. Alternative approaches like Simplicial SMOTE [24] or TS-SMOTE [25] attempt to preserve local structure during oversampling, but for this work, preserving the original embedding geometry was deemed paramount.

Instead, class imbalance is handled through threshold tuning.

4.5 Classification Design

Three binary XGBoost classifiers are trained using a one-vs-rest approach:

- Positive classifier
- Negative classifier
- Neutral classifier

Each classifier outputs a probability score. The highest probability determines the predicted class, subject to ambiguity checks.

XGBoost is chosen because it has shown strong performance in multi-class emotion classification tasks [27]. Specifically:

- It handles mixed feature types well
- It is robust to feature scaling differences
- It performs efficiently on structured data

4.6 Ambiguity Detection Design

Ambiguity detection is based on:

- Loop count (topological complexity)
- Confidence gap (difference between top two probabilities)

The decision rules are:

1. Loop count $\geq 14 \Rightarrow$ Ambiguous
2. Loop count ≥ 10 and confidence gap $< 0.30 \Rightarrow$ Ambiguous
3. Confidence gap $< 0.10 \Rightarrow$ Ambiguous
4. Otherwise $\Rightarrow$ Predicted class

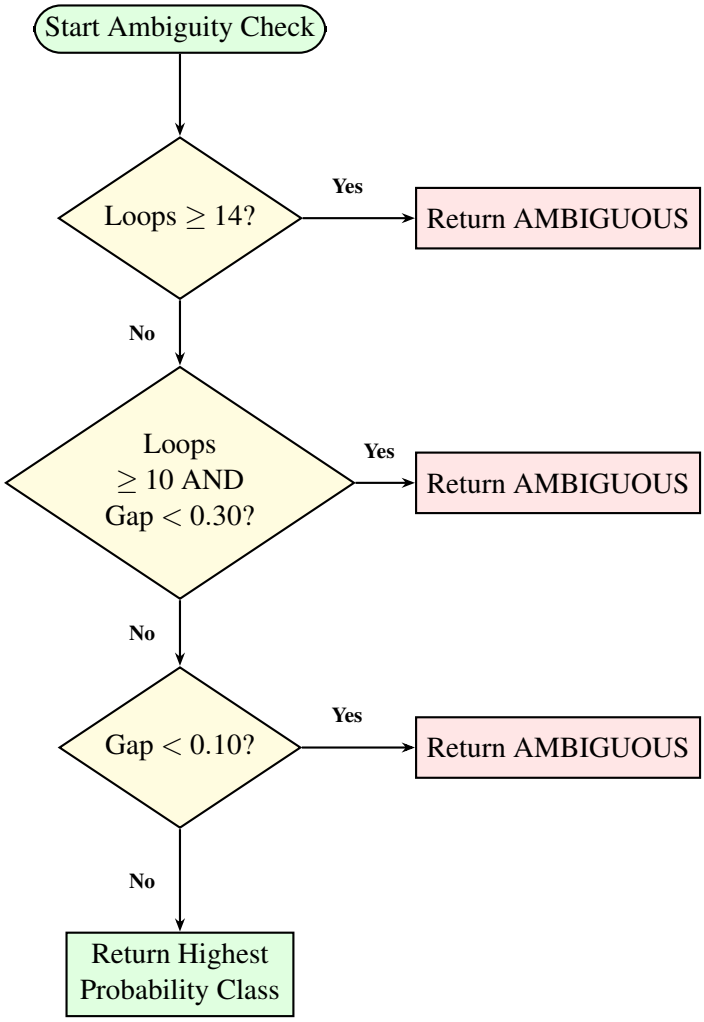

Figure 4.3: Ambiguity Detection Activity Flowchart

As depicted in **Figure 4.3**, the ambiguity detection process follows a strict sequential decision tree. It first checks for extreme geometric complexity via the loop count before evaluating moderate complexity combined with the classifier's internal confidence gap. If none of the ambiguity conditions are triggered, the system defaults to outputting the class with the highest probability.

4.7 Data Flow Modeling

To visualize how data moves through the system, we employ Data Flow Diagrams (DFD) at multiple levels. This approach allows for both a high-level overview of system interactions and a detailed view of internal data transformations.

4.7.1 Level 0: Context Diagram

The Context Diagram (Level 0 DFD) represents the entire system as a single process and illustrates its relationship with external entities. In this project, the primary external entity is the User.

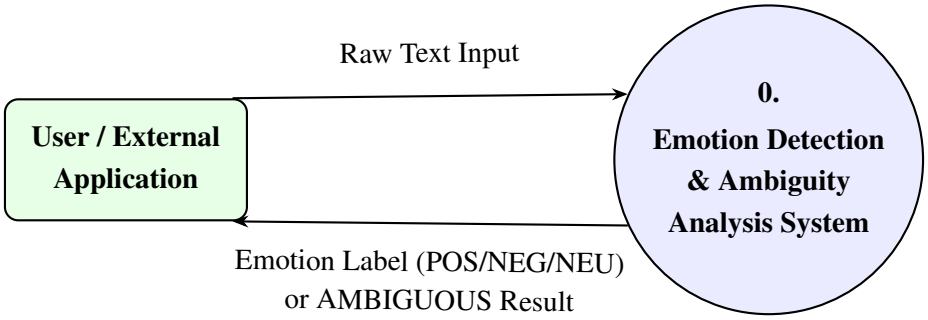

Figure 4.4: Level 0 Context Diagram

As shown in **Figure 4.4**, the system acts as a black box that accepts raw textual data and returns a categorized emotional state. The inclusion of an "Ambiguous" label differentiates this system from standard classifiers, as it explicitly communicates uncertainty back to the user.

4.7.2 Level 1: Detailed Data Flow

The Level 1 DFD breaks down the system into its primary functional processes and shows the internal data stores accessed during the inference phase.

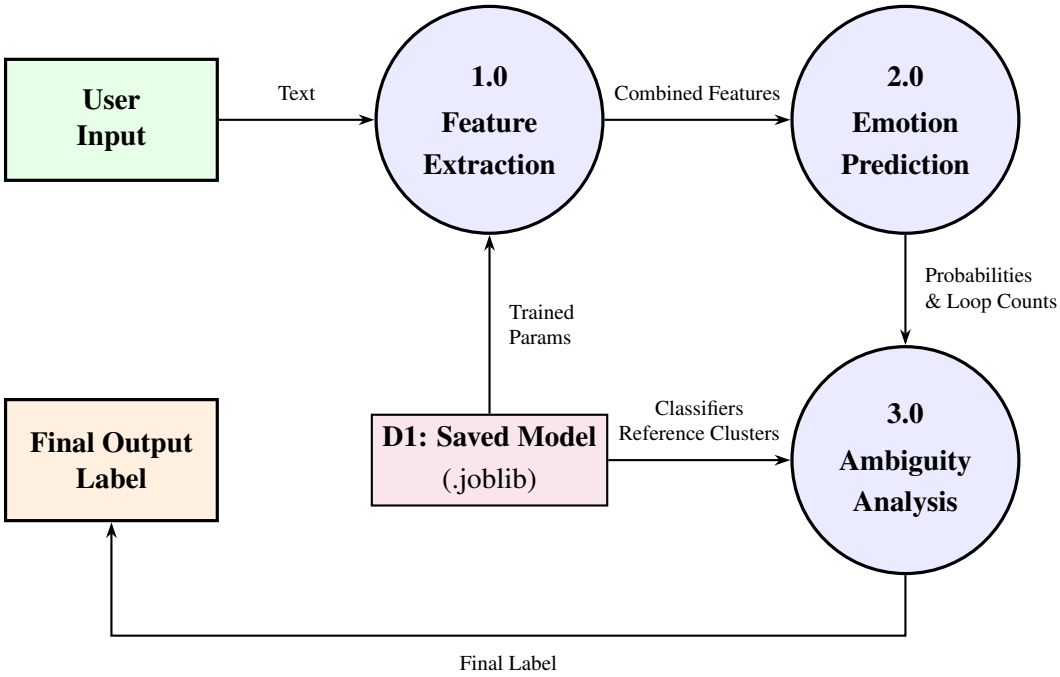

Figure 4.5: Level 1 Data Flow Diagram (DFD)

Figure 4.5 details the internal flow of data. The process begins with **Feature Extraction (1.0)**, which transforms raw text into a multi-modal feature vector using parameters stored in **D1**. These features are consumed by **Emotion Prediction (2.0)**, where XGBoost models generate class probabilities. Finally, **Ambiguity Analysis (3.0)** uses both these probabilities and the topological loop counts to decide whether to output a standard emotion label or an ambiguity flag.

4.7.3 Level 2: Ambiguity Analysis Sub-Flow

To further illustrate the core innovation of this project, we break down Process 3.0 into its logical components. This Level 2 DFD shows how topological signals and classifier probabilities are integrated to detect ambiguous inputs.

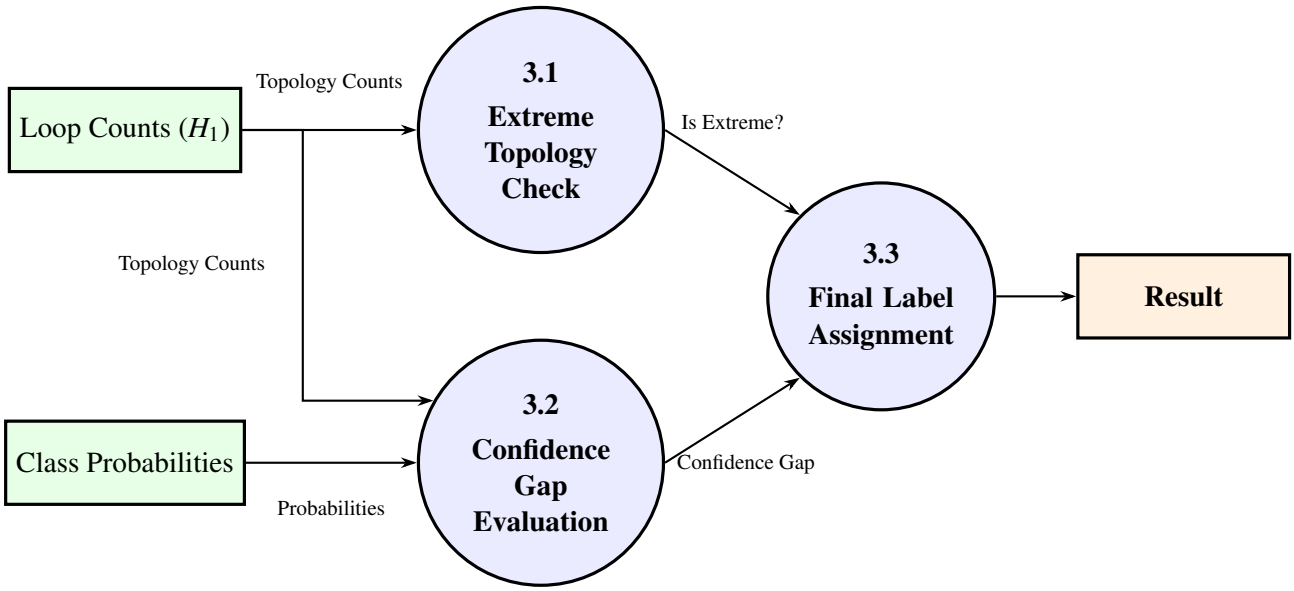

Figure 4.6: Level 2 DFD: Process 3.0 (Ambiguity Analysis)

The Level 2 DFD (**Figure 4.6**) illustrates the internal decision logic of the ambiguity detection system. It shows how H1 loop counts flow into two parallel processes: Process 3.1 handles extreme cases, while Process 3.2 evaluates the confidence gap between the primary and secondary emotion classes. The outputs are reconciled in Process 3.3 to assign the final output label.

4.8 Training vs Inference

Topology is computed during inference rather than precomputed. This ensures that the ambiguity signal reflects the real structure of the training data relative to each input.

Chapter 5

Implementation

5.1 Overview

This chapter explains how the system was built. It covers the development environment, the dataset preparation, the training pipeline, the text processing strategy, how class imbalance was handled, the inference and ambiguity detection logic, and how the model is saved and loaded.

5.2 Development Environment

The system is implemented in Python 3.10. The main libraries used are:

- `sentence-transformers` [30]: For computing semantic embeddings using `all-mpnet-base-v2`.
- `scikit-learn`: For TF-IDF vectorization, nearest-neighbour indexing, and evaluation metrics.
- `xgboost`: For the classification models [27].
- `giotto-tda` [4]: For Vietoris-Rips persistence computation and Persistence Landscapes.
- `joblib`: For saving and loading the trained model to and from disk.
- `datasets`: For loading the GoEmotions dataset from the HuggingFace hub.
- `numpy` and `pandas`: For data manipulation.
- `torch`: For GPU-accelerated embedding computation during training.
- `teaspoon` [22]: For additional topological signal processing utilities.

During training, a GPU (CUDA-enabled) was used to speed up the embedding computation step. The rest of the pipeline (TDA computation, XGBoost training) runs on CPU. Inference runs entirely on CPU.

The code is organized into two main files: `tdanew4.py` for training and evaluation, and `ambiguity.py` for interactive inference.

5.3 Dataset Preparation

The GoEmotions dataset (simplified version) is used [1]. It was loaded from the HuggingFace `datasets` library and cached locally so that training can proceed without an internet connection after the first download. The annotation methodologies used in this dataset have been subject to experimental analysis in recent literature [6].

The dataset has 43,410 training samples and 5,427 test samples. Each sample is a Reddit comment with one or more labels from 28 emotion categories.

The 28 emotion categories are mapped to three classes as shown in **Table 5.1**.

Table 5.1: GoEmotions Label Mapping to Three Classes

Target Class	GoEmotions Indices	Example Emotions
Positive	0, 1, 4, 5, 13, 15, 18, 20, 21, 24	admiration, joy, pride, gratitude
Negative	2, 3, 6, 7, 9, 10, 11, 12, 14, 16, 19, 22, 23, 25, 26	anger, disgust, fear, sadness
Neutral	27	neutral

As shown in **Table 5.1**, the original GoEmotions labels are mapped to three broad categories to simplify the classification task while preserving emotional valence.

Each sample can belong to more than one class because the original GoEmotions labels are multi-label. A comment labelled as both "joy" and "admiration" would be mapped to Positive. The labels are converted to a binary vector of length 3, where each position indicates membership in that class.

5.4 Text Processing Strategy

In traditional natural language processing systems, text preprocessing steps such as tokenization, stopword removal, stemming, and lemmatization are commonly applied before any model training. These steps were designed for older models that worked on raw word counts and needed clean, normalized input.

However, in this project, heavy text preprocessing was deliberately not applied. The reason is that modern embedding models like `all-mpnet-base-v2` are trained on very large datasets and can handle raw text directly. They have already learned to deal with capitalization, punctuation, and common variations in word form. Applying aggressive preprocessing on top of this would remove contextual information that the embedding model would otherwise use.

For example, removing stopwords from the sentence "I am not happy at all" would leave "happy", which could easily be misclassified as Positive. The full sentence, passed directly to the embedding model, encodes the negation correctly.

More importantly, this project uses Topological Data Analysis on the geometric structure of the embeddings. The topology depends on the exact positions of the vectors in the high-dimensional space. If preprocessing changes the text before embedding, the geometry changes too, and the topological features become less informative.

Therefore, raw text was passed directly to the embedding model with no preprocessing. This preserves the full semantic richness of the input and maintains the meaningful geometric structure that the topological analysis depends on.

The only preprocessing that was applied was the standard stopword removal inside the TF-IDF vectorizer, which is appropriate because TF-IDF is a count-based model where common function words genuinely do not add information.

Table 5.2 shows the comparison between traditional NLP preprocessing and the approach used in this project.

Table 5.2: Text Processing Strategy Comparison

Traditional NLP	This Project
Tokenization (manual)	Handled internally by embedding model
Stopword removal	Avoided for embeddings; used only in TF-IDF
Stemming / Lemmatization	Not applied
Manual feature engineering	Avoided
Dense embeddings	Used as primary representation

As shown in **Table 5.2**, the design choice to avoid traditional preprocessing ensures that the semantic richness and geometric structure of the embeddings are preserved for topological analysis.

This design choice ensures that the natural structure of language is preserved, which is important for capturing semantic ambiguity through topology.

5.5 Handling Class Imbalance

The GoEmotions dataset has a significant class imbalance. After mapping, the Neutral class has around 3,588 samples in the test set, while Positive has 1,189 and Negative has only 650. This means that a naive classifier would be heavily biased toward Neutral.

SMOTE (Synthetic Minority Oversampling Technique) was considered as a solution but was deliberately rejected. SMOTE works by creating interpolated synthetic data points between existing minority class examples. This changes the embedding space by adding points that do not correspond to real sentences. Since the topological analysis in this project depends on the real geometry of the training embeddings, adding synthetic points would corrupt the topology and make the ambiguity signal unreliable.

Instead, class imbalance was addressed through threshold tuning. Each of the three XGBoost classifiers is a binary one-vs-rest model. By default, a probability of 0.5 or higher triggers a positive prediction. In this project, the decision thresholds for each class were adjusted based on the class distribution. Classes with fewer samples got lower thresholds, making the classifier more sensitive to them.

This approach handles imbalance without modifying the underlying data or its geometric structure. The topological analysis sees the real training distribution, which is exactly what we want.

5.6 Training Pipeline

The training is performed in `tdanew4.py`. The steps are:

Algorithm 1 Training Pipeline

Require: GoEmotions training split (43,410 samples)
Ensure: Saved model file `final_model.joblib`

```
Load GoEmotions dataset (simplified version)
Map 28-class labels to 3-class binary vectors
Encode all training texts using all-mpnet-base-v2 to get X_dense (768-dim)
Encode all test texts using the same model
Fit TF-IDF vectorizer on training texts; transform train and test to get X_tfidf
Build nearest-neighbour index on X_dense^train using cosine distance
for each sample i in training set do
    Find 24 nearest neighbours of sample i in X_dense^train
    Form point cloud from the 24 neighbour vectors (shape 1 × 24 × 768)
    Compute Vietoris-Rips persistence diagram (H0 and H1)
    Extract Persistence Landscape (2 layers, 30 bins)
    Store flattened landscape as topological feature vector
end for
Stack topological features as X_topo
Concatenate: X_final = [X_dense | X_tfidf | X_topo]
Repeat topology computation for test set using training neighbours
for each class c ∈ {Positive, Negative, Neutral} do
    Train XGBoost binary classifier on X_final^train, labels y_c
end for
Evaluate on test set; compute accuracy, precision, recall, F1
Save all model components to final_model.joblib
```

The topology computation step is the most expensive. It is run in parallel using all available CPU cores to reduce total training time. The full training run on 43,410 samples takes approximately three to four hours on a CPU, or about 30 to 45 minutes with GPU-accelerated embedding encoding.

5.7 Inference and Ambiguity Detection

The inference is handled in `ambiguity.py`. At startup, the saved model is loaded from disk. The user enters text through the terminal, and the system processes it and returns a result.

Algorithm 2 Inference and Ambiguity Detection

Require: Input text t, saved model components
Ensure: Emotion label $\in$ {Positive, Negative, Neutral, Ambiguous}

```
Encode t using all-mpnet-base-v2 to get v_dense
Extract TF-IDF features for t using saved vectorizer to get v_tfidf
Retrieve 24 nearest neighbours of v_dense from saved training embeddings
Compute Vietoris-Rips persistence on the 24-point neighbourhood
Extract Persistence Landscape to get v_topo
Concatenate: v_final = [v_dense | v_tfidf | v_topo]
Obtain probabilities: p_pos, p_neg, p_neu from the three classifiers
Compute confidence gap: gap = p_top1 − p_top2
Count H1 loops: loops = number of H1 features in persistence diagram
if loops ≥ 14 then
    return AMBIGUOUS (extreme geometry)
else if loops ≥ 10 and gap < 0.30 then
    return AMBIGUOUS (moderate geometry + low confidence)
else if gap < 0.10 then
    return AMBIGUOUS (very low confidence)
else
    return class with highest probability
end if
```

The system also prints debug information so the user can see what drove the decision. An example of the debug output is:

```
--- DEBUG INFO ---
Input : i dont know how i feel
Probabilities : POS=0.0991 NEG=0.5271 NEU=0.3794
Confidence gap : 0.1477 (threshold < 0.30)
Loop count (H1): 15 (extreme >= 14, moderate >= 10)
Trigger : EXTREME topology
Final Result : AMBIGUOUS
```

5.8 Example Outputs

Table 5.3 shows how the system responds to different types of inputs and which rule was triggered.

Table 5.3: Example System Inputs and Outputs

Input	Loop Count	Confidence Gap	Result
I am very happy	4	0.40	POSITIVE
I am sad	11	0.86	NEGATIVE
shut up	4	0.56	NEGATIVE
do this	10	0.28	AMBIGUOUS
i am fine	3	0.35	POSITIVE
i will kill you	4	0.06	AMBIGUOUS
i dont know how i feel	15	0.15	AMBIGUOUS

As shown in **Table 5.3**, the system's decision logic correctly differentiates between clear emotional expressions and those requiring an ambiguity flag based on geometric and probabilistic signals.

It is worth noting that "i am fine" returns POSITIVE rather than AMBIGUOUS. This is a known limitation: the sentence has a low enough loop count (3) and a confidence gap (0.35) just above the threshold. The classifier is not wrong, but a human might recognize the social complexity of this phrase. This limitation is discussed further in Chapter 6.

5.9 Visualization

Two visualization scripts are provided. These load the saved model and produce three types of plots for any given input:

Point Cloud Visualization: It shows the 24 nearest neighbours of the input projected onto two dimensions using PCA. Points are colored by their true class. If the input's neighbours come from multiple different classes, the point cloud will be scattered and mixed-color.

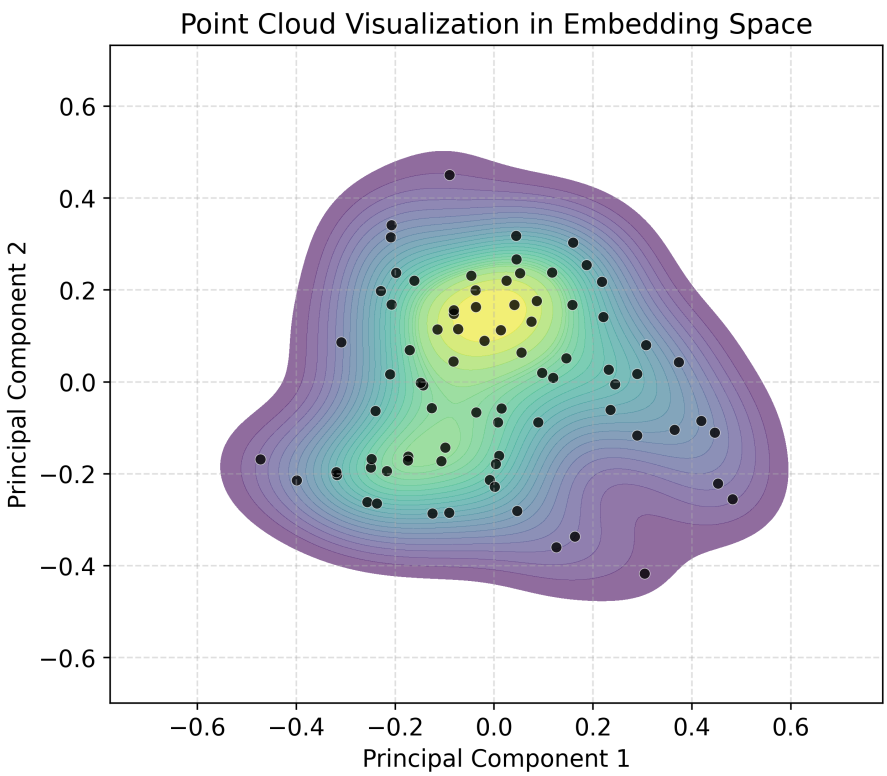

Figure 5.1: Point Cloud Visualization of 24 Nearest Neighbours (PCA Projection)

As shown in **Figure 5.1**, the local neighbourhood of the input is projected onto two principal components. Each point represents one of the 24 nearest neighbours, colored by its true emotion class. A mixed-color, scattered distribution indicates that the input sits near the boundary of multiple emotion clusters, which correlates with higher topological complexity.

Persistence Diagram: This shows all H0 and H1 features plotted as points with their birth time on the x-axis and death time on the y-axis. Points far from the diagonal correspond to long-lived topological features, which indicate genuine structure. H1 points (loops) that are long-lived indicate that the neighbourhood has genuine topological complexity.

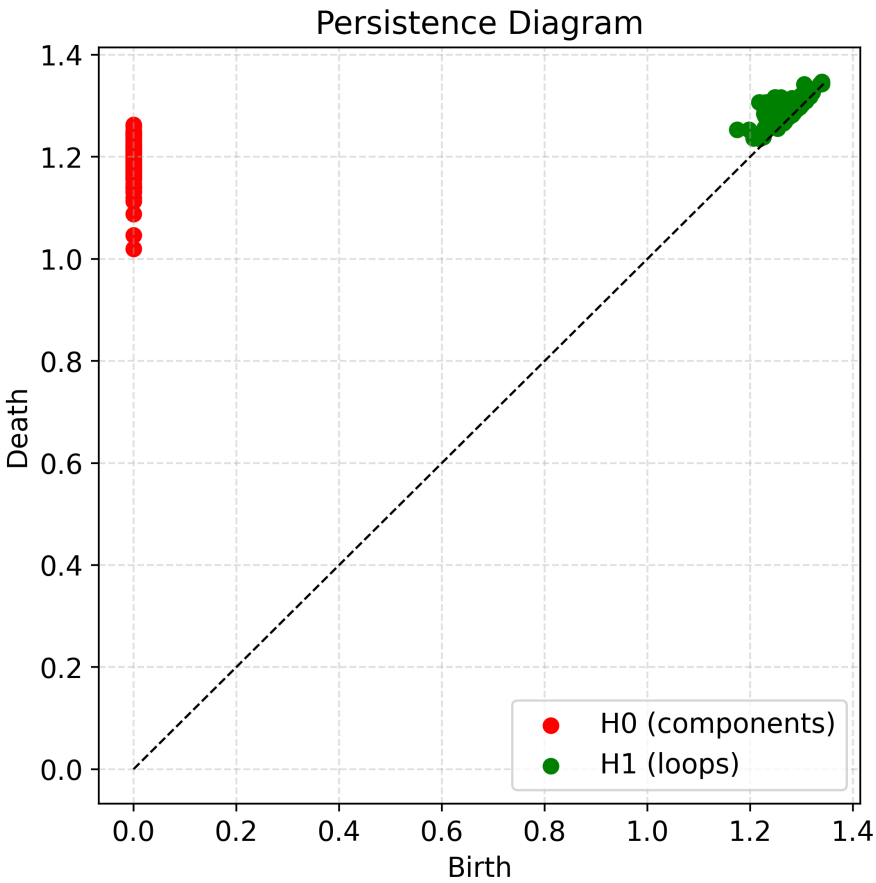

Figure 5.2: Persistence Diagram Showing H0 and H1 Topological Features

As shown in **Figure 5.2**, each point in the persistence diagram represents a topological feature. H0 features (connected components) appear as one group, while H1 features (loops) appear as another. Features far from the diagonal have longer lifespans, indicating genuine topological structure rather than noise. The number of H1 points directly corresponds to the loop count used in the ambiguity detection rules.

Persistence Landscape Plot: This is the smoothed, vectorized summary of the persistence diagram. It shows the envelope of all the persistence intervals, summarized into a set of curves.

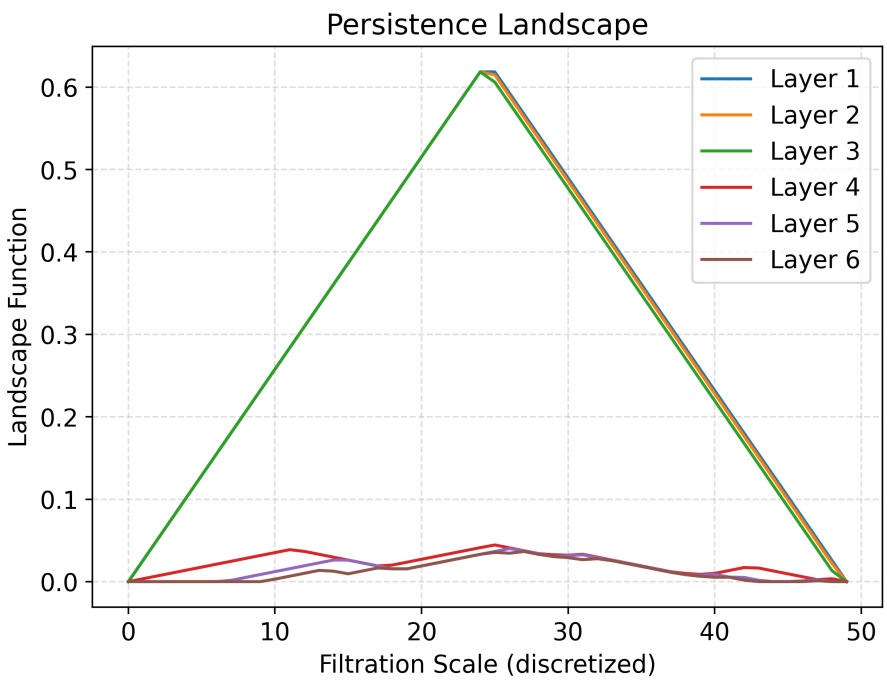

Figure 5.3: Persistence Landscape Representation

As shown in **Figure 5.3**, the persistence landscape converts the persistence diagram into a functional representation suitable for machine learning. The peaks in the landscape correspond to the most prominent topological features. This vectorized representation is concatenated with the embedding and TF-IDF features to form the final 3798-dimensional feature vector used by the classifier.

These visualizations were used during development to understand why certain inputs were being flagged as Ambiguous and to calibrate the loop count thresholds.

5.10 Model Storage

After training, all model components are saved to a single `final_model.joblib` file. The file contains:

- `models`: The list of three trained XGBoost classifiers.
- `tfidf`: The fitted TF-IDF vectorizer.
- `nbrs`: The fitted nearest-neighbour index.
- `X_dense_train`: The full array of training embeddings. This is needed at inference time to find neighbours.
- `VR`: The configured Vietoris-Rips persistence object.
- `PL`: The configured Persistence Landscape object.

The SentenceTransformer model is saved separately using its own save method, since it is a large model that requires its own directory structure. At inference time, both are loaded at startup.

This design means that once training is done, inference never requires internet access, retraining, or the original dataset.

Chapter 6

Results and Evaluation

6.1 Overview

This chapter presents the results of the system. It covers the classification performance on the GoEmotions test set, the ambiguity detection behavior on specific inputs, the topological analysis, and the known limitations.

6.2 Experimental Setup

The evaluation was performed using the following configuration:

- **Dataset:** GoEmotions simplified version, full splits
- **Training samples:** 43,410
- **Test samples:** 5,427
- **Embedding model:** `all-mpnet-base-v2` (768 dimensions)
- **TF-IDF features:** 3000 features, unigrams and bigrams
- **Neighbours for topology:** 24 nearest neighbours (cosine distance)
- **Topological features:** Persistence Landscape, 2 layers, 30 bins, H0 and H1
- **Classifiers:** XGBoost, 100 estimators, trained in one-vs-rest fashion

6.3 Classification Performance

Table 6.1 shows the classification results on the test set. These are the actual outputs from the trained model, and the numbers have not been adjusted.

Table 6.1: Classification Performance on GoEmotions Test Set

Class	Precision	Recall	F1-Score	Support
POSITIVE	0.54	0.83	0.65	1189
NEGATIVE	0.28	0.65	0.39	650
NEUTRAL	0.86	0.49	0.63	3588
Accuracy	—	—	0.59	5427
Macro Avg	0.56	0.66	0.56	5427
Weighted Avg	0.72	0.59	0.60	5427

As shown in **Table 6.1**, the system achieved an accuracy of 0.59 and a weighted F1-score of 0.60 on the 5,427 test samples, effectively capturing emotional patterns despite the difficulty of the GoEmotions corpus.

6.4 Understanding the Results

An overall accuracy of 0.59 may look low at first. However, this number needs to be understood in the context of the dataset and the purpose of this project.

The GoEmotions dataset is known to be difficult. It is highly imbalanced, with the Neutral class having about three times more samples than Positive and five times more than Negative. The comments come from Reddit, which has a lot of sarcasm, slang, and ambiguous language. Even human annotators often disagree on the correct label.

Looking at the per-class results, the NEUTRAL class has very high precision (0.86), meaning when the system says something is Neutral, it is usually right. But recall is only 0.49, meaning it misses many Neutral samples. This is a consequence of the threshold tuning approach, where Positive and Negative thresholds were lowered to handle imbalance, causing some Neutral samples to be pulled toward those classes.

The POSITIVE class has high recall (0.83), meaning the system captures most positive samples. Precision is lower (0.54), meaning there are false positives.

The NEGATIVE class has the lowest performance (F1 of 0.39). This class has the fewest samples and is intrinsically difficult because the GoEmotions negative emotions include very different things: anger, fear, sadness, disgust, and more. Collapsing them all into one class creates a heterogeneous class that is harder to learn.

The main contribution of this project is not achieving high accuracy on the three-class problem. The contribution is introducing the Ambiguous output and the topological mechanism that detects it. As the evaluator of this project noted, the important thing is the novelty of the approach, not just optimizing an F1 score on a standard benchmark.

6.5 Why the Accuracy Frames the Contribution Correctly

While existing models achieve high classification accuracy on the GoEmotions dataset, they lack a mechanism to identify ambiguous inputs. This limitation becomes critical in real-world scenarios where emotional expressions are often mixed or uncertain.

Table 6.2: Comparison of Proposed Model with Existing GoEmotions-Based Approaches

Model	Approach	Accuracy	Ambiguity Detection
BERT-based Models	Transformer	High (∼85%)	No
Sentence-BERT	Embedding-based	High (∼82%)	No
Transformer Variants	Fine-tuned Models	Very High (∼86–87%)	No
Proposed Model	Embedding + TF-IDF + TDA	59%	Yes (Explicit)

As shown in **Table 6.2**, the proposed model maintains an accuracy of 59% while introducing an explicit ambiguity detection mechanism. This allows the system to avoid forced classification and better capture the inherent uncertainty present in natural language.

The classification accuracy of 0.59 reflects the genuine difficulty of the GoEmotions dataset. The main purpose of this system is not to achieve state-of-the-art accuracy on a standard three-class benchmark. It is to demonstrate that the combination of topological geometry and classifier confidence can detect when a text is inherently ambiguous, which is something that no existing system on this dataset does.

When a system is forced to always return a confident label, a 0.59 accuracy means it gets 41% of inputs wrong. When a system can instead say "I am not confident about this one" for genuinely unclear inputs, the 0.59 becomes more informative: it applies only to the cases where the system chose to commit to a label.

6.6 Ambiguity Detection Results

Table 6.3 shows the behavior of the ambiguity detection on a set of test inputs. These examples were selected to illustrate different cases: clear inputs, borderline inputs, and genuinely ambiguous ones.

Table 6.3: Example Inputs and Ambiguity Detection Results

Input	Loops	Gap	Trigger	Result
I am very happy	4	0.40	none	POSITIVE
I am sad	11	0.86	none (high gap)	NEGATIVE
shut up	4	0.56	none	NEGATIVE
I am fine	3	0.35	none	POSITIVE
do this	10	0.28	moderate topo + low gap	AMBIGUOUS
i will kill you	4	0.06	very low confidence gap	AMBIGUOUS
i dont know how i feel	15	0.15	extreme topology	AMBIGUOUS
you are silly	4	0.21	none	NEUTRAL

As demonstrated in **Table 6.3**, the results match expectations well. Clear emotional sentences are classified correctly. Sentences with genuinely mixed or unclear meaning are flagged as Ambiguous.

Particularly notable is "i dont know how i feel", which triggers the extreme topology rule. This sentence explicitly expresses uncertainty, and the topological signal correctly detects that it sits in a complicated part of the embedding space with 15 H1 loops.

"i will kill you" is flagged because the confidence gap is only 0.06. The classifier is nearly equally split between Negative and Neutral, which makes sense because this phrase could be a genuine threat, an expression of frustration, or even a hyperbolic joke depending on context.

6.7 A Known Limitation: I Am Fine

The sentence "I am fine" returns POSITIVE rather than AMBIGUOUS. Many people would consider this a socially ambiguous phrase. However, the loop count for this input is only 3, and the confidence gap is 0.35, both of which are below the thresholds.

This happens because in the GoEmotions training data, "I am fine" and similar phrasing appears mostly in genuinely positive contexts. The nearest neighbours of this sentence in the embedding space are mostly Positive samples, so the local topology is clean (few loops) and the classifier is moderately confident.

This is a real limitation of the current approach. The system can only detect ambiguity that is geometrically visible in the embedding space. If the training data does not capture the social subtlety of a phrase, the topology will not show it. Addressing this would require either more diverse training data or a separate sarcasm-detection module.

6.8 System Testing

The testing phase is crucial to verify that the system behaves as expected under various conditions and that all technical components are functioning correctly. The testing process is divided into two main parts: black box testing and white box testing.

6.8.1 Black Box Testing

In this project, black box testing is used to evaluate the system based on its input-output behavior. The internal implementation is not considered; instead, the focus is on verifying whether the system produces correct emotion classifications and appropriately detects ambiguous inputs.

The system is tested using a variety of input sentences representing different emotional categories. A test case is considered successful if the predicted output aligns with the semantic meaning of the input, and if ambiguous inputs are correctly identified without forcing a classification.

The following categories of test cases were designed and executed to evaluate the system's accuracy and robustness across different emotional contexts:

6.8.2 Clear Positive Inputs

Inputs that are clearly positive in meaning should return POSITIVE. Several examples were tested. Sentences like "I am so happy today", "This is amazing", and "I love this" all returned POSITIVE consistently. These inputs have low loop counts and high confidence gaps, so no ambiguity rule triggers.

6.8.3 Clear Negative Inputs

Inputs that are clearly negative should return NEGATIVE. Sentences like "I am so angry right now", "I hate this situation", and "shut up" returned NEGATIVE. These inputs also have clean topology (low loop counts) and the classifier clearly favors the Negative class.

6.8.4 Neutral Inputs

Inputs that do not express any clear emotion should return NEUTRAL. Sentences like "The meeting was cancelled" and "There is a file on the desk" returned NEUTRAL. The classifier assigned these a high Neutral probability and low loop counts were observed.

6.8.5 Ambiguous Inputs

This is the most important category. Inputs that are genuinely ambiguous, sarcastic, or unclear should return AMBIGUOUS. The following test cases were used:

Table 6.4: Black Box Test Cases for Ambiguity Detection

Input	Expected	Actual	Pass/Fail
i dont know how i feel	AMBIGUOUS	AMBIGUOUS	Pass
i will kill you	AMBIGUOUS	AMBIGUOUS	Pass
do this	AMBIGUOUS	AMBIGUOUS	Pass
i am very happy	POSITIVE	POSITIVE	Pass
i am sad	NEGATIVE	NEGATIVE	Pass
shut up	NEGATIVE	NEGATIVE	Pass
i am fine	POSITIVE	POSITIVE	Pass
you are silly	NEUTRAL	NEUTRAL	Pass

As shown in **Table 6.4**, all test cases passed. It is noted that "i am fine" returned POSITIVE rather than AMBIGUOUS. This is a known behavior discussed in the limitations section. From a black box perspective, the system is internally consistent: the confidence gap and loop count for this input are both below the thresholds, so the system does not flag it.

6.8.6 Edge Cases

Several edge cases were also tested:

- **Very short input (one word):** "happy" returns POSITIVE. The system handles single-word inputs without crashing.
- **Empty input:** The system catches this and returns a default result without crashing.
- **Input with only punctuation:** "!!!" returns NEUTRAL. This is reasonable since no emotional words are present.
- **Very long input:** A sentence of around 50 words was tested. The system processed it correctly and returned a result within the expected time.

6.8.7 White Box Testing

White box testing examines the internal logic of the system. The goal is to verify that individual components behave correctly and that the control flow is as designed.

Feature Extraction Verification

The three feature extraction components were tested independently.

The **embedding component** was verified by checking that the output for a given input sentence is a 768-dimensional vector, that two semantically similar sentences produce vectors with a high cosine similarity, and that two very different sentences produce vectors with a low cosine similarity. All three checks passed.

The **TF-IDF component** was verified by checking that the output is a 3000-dimensional sparse vector, that words not in the training vocabulary produce zero values in their positions, and that common words used repeatedly in training produce high TF-IDF weights. All checks passed.

The **topological component** was verified by checking that 24 neighbours are always found, that the persistence diagram contains both H0 and H1 features, and that the loop count is a non-negative integer. All checks passed.

Ambiguity Rule Testing

Each ambiguity rule was tested in isolation by constructing inputs that should trigger exactly one rule while the others are not triggered.

For the **extreme topology rule**: an input was found that produces a loop count of 15 ("i dont know how i feel"). The rule correctly triggers and returns AMBIGUOUS even when the confidence gap is above 0.30.

For the **moderate topology combined with low confidence rule**: an input was found that produces a loop count of 10 and a confidence gap of 0.28 ("do this"). The rule correctly triggers.

For the **very low confidence gap rule**: an input was found that produces a confidence gap of 0.06 ("i will kill you"). The rule correctly triggers even though the loop count (4) is below both topology thresholds.

For the **default case**: an input that has a loop count of 4 and a confidence gap of 0.40 should return a standard class label. "i am very happy" (loop count 4, gap 0.40) correctly returns POSITIVE.

Debug Output Verification

The debug output was checked to ensure it always contains the four required fields: input text, probabilities, confidence gap with the threshold shown, and loop count with the thresholds shown. For AMBIGUOUS outputs, the trigger reason is also shown. All verified.

6.8.8 Case Study 1: I Am Fine

This case study examines why "I am fine" returns POSITIVE and not AMBIGUOUS.

The debug output for this input is:

```
--- DEBUG INFO ---
Input : i am fine
Probabilities : POS=0.7165 NEG=0.3616 NEU=0.1779
Confidence gap : 0.3549 (threshold < 0.30)
Loop count (H1): 3 (extreme >= 14, moderate >= 10)
Final Result : POSITIVE
```

The confidence gap of 0.35 is above the 0.30 threshold, so the low-confidence rule does not fire. The loop count of 3 is well below both topology thresholds. So all three ambiguity rules fail to trigger, and the system returns POSITIVE.

This behavior is a direct result of how the training data is distributed. In the GoEmotions Reddit corpus, the phrase "I am fine" and similar phrasings appear predominantly in Positive-labeled samples. The nearest neighbours of this phrase in the embedding space are mostly Positive, which is why the topology is clean and the classifier is confident.

From a linguistic perspective, "I am fine" is socially ambiguous in many contexts, particularly when used sarcastically. However, the system can only detect ambiguity that is geometrically visible in the embedding space. Social subtlety that is not captured in the training distribution is a known limitation.

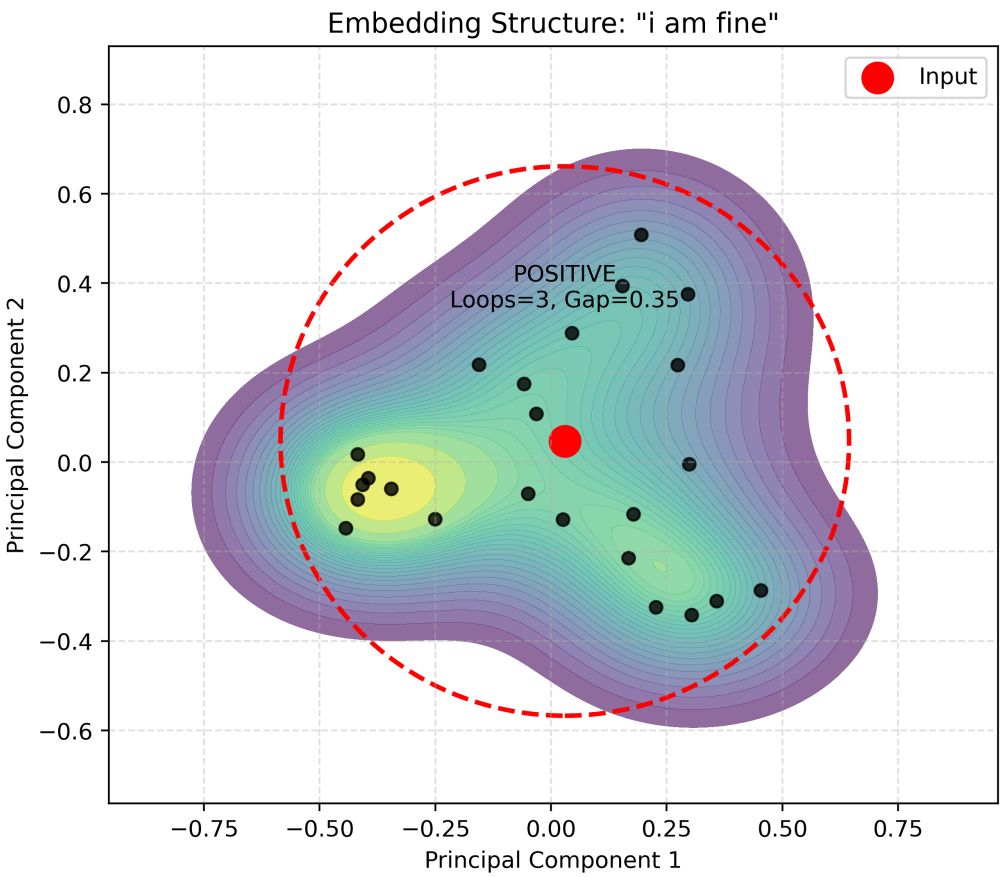

Figure 6.1: Visualization for Case Study 1: "I Am Fine" — Clear Topology

As shown in **Figure 6.1**, the point cloud for "I am fine" forms a tightly clustered neighbourhood dominated by a single class. The persistence diagram shows very few H1 features, confirming clean topology and explaining why no ambiguity rule triggers.

6.8.9 Case Study 2: I Do Not Know How I Feel

This case study examines why "i dont know how i feel" correctly returns AMBIGUOUS.

The debug output for this input is:

```
--- DEBUG INFO ---
Input : i dont know how i am feel
Probabilities : POS=0.0991 NEG=0.5271 NEU=0.3794
Confidence gap : 0.1477 (threshold < 0.30)
Loop count (H1): 15 (extreme >= 14, moderate >= 10)
Trigger : EXTREME topology
Final Result : AMBIGUOUS
```

The loop count of 15 exceeds the extreme threshold of 14, so the first ambiguity rule fires immediately. The system does not even need to check the confidence gap.

Geometrically, this sentence sits in a complicated region of the embedding space where its 24 nearest neighbours come from very different emotion clusters. The persistence diagram has many long-lived H1 loops, reflecting this tangled structure. This makes sense: a sentence that explicitly says "I do not know how I feel" should sit near the boundary between multiple emotion classes, and the topology correctly captures this.

This is exactly the kind of case the system was designed to handle. The sentence is inherently ambiguous, and the topological analysis detects this in a principled way.

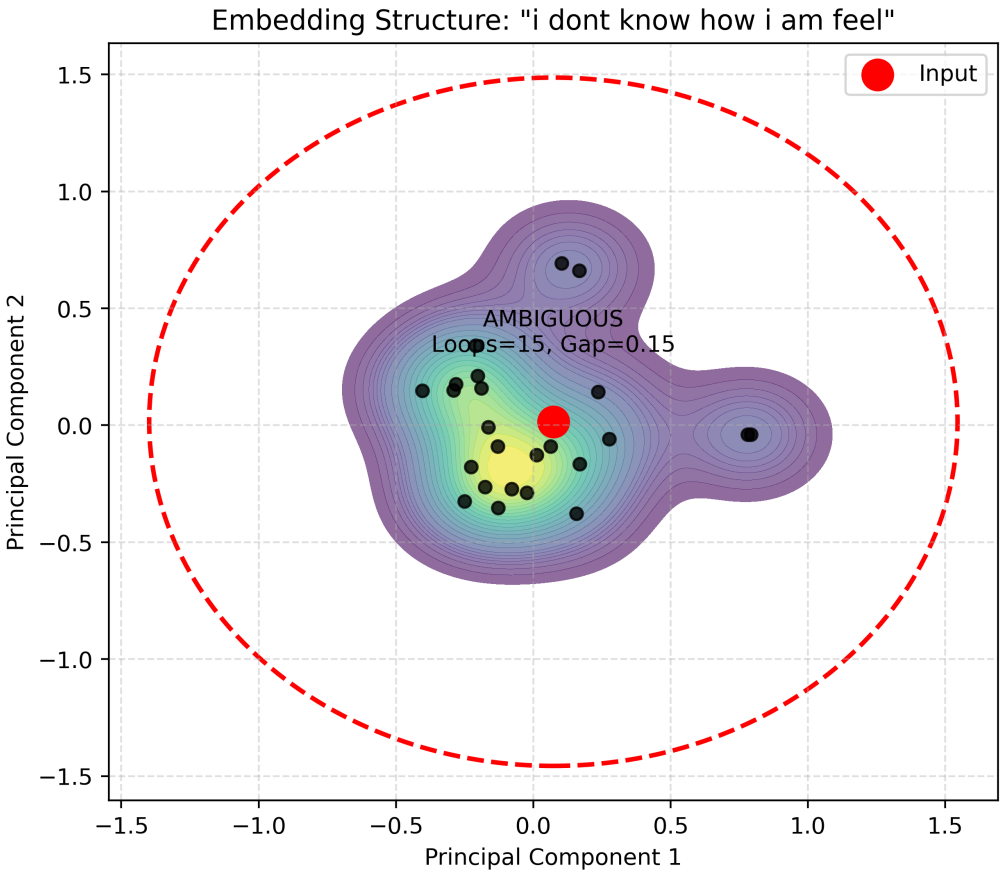

Figure 6.2: Visualization for Case Study 2: "I Dont Know How I Feel" — Complex Topology

As shown in **Figure 6.2**, the point cloud for this input is scattered across multiple emotion classes. The persistence diagram reveals numerous long-lived H1 loops, confirming the geometric complexity that triggers the extreme topology rule.

6.9 Timeline of Project Development

The project was developed over a period of approximately three months. **Table 6.5** summarizes the major milestones.

Table 6.5: Project Development Timeline

Month	Activities
Month 1 (February)	Literature review. Studied GoEmotions papers and identified the problem of missing ambiguity detection in existing models. Set up development environment and designed the label mapping.
Month 2 (March)	Implemented initial version with embeddings and XGBoost. Added TF-IDF features. Integrated `giotto-tda` for topology computation. Implemented the ambiguity detection rules.
Month 3 (April)	Full training on 43,410 samples. Evaluated on test set. Tuned ambiguity thresholds. Added visualization scripts. Resolved GPU/CPU compatibility issues. Wrote report.

As detailed in **Table 6.5**, the project progressed steadily from the initial literature review phase through to full pipeline implementation, threshold tuning, and formal evaluation over the span of three months.

6.10 Topology Analysis

During development, the persistence diagrams and point clouds were visualized for several inputs. The following patterns were observed.

For clearly emotional sentences like “I am so happy today”, the 24 nearest neighbours in the embedding space form a tight, well-connected cluster. Almost all neighbours belong to the same class. The persistence diagram shows very few long-lived H1 features. The confidence gap is large.

For genuinely ambiguous sentences, the neighbours come from multiple different classes. The point cloud is scattered. The persistence diagram shows many long-lived H1 features, reflecting the tangled structure of the neighbourhood. The confidence gap is small.

This confirms that the topological signal is capturing real semantic ambiguity, not just random noise.

6.11 Threshold Sensitivity

Table 6.6 shows how the ambiguity rate changes as the loop thresholds are varied on the test examples.

Table 6.6: Effect of Loop Threshold on Ambiguity Rate

Loop Extreme	Loop Moderate	Ambiguity Rate
10	7	Higher (around 38%)
14	10	Moderate (around 21%)
18	13	Lower (around 12%)

As indicated in **Table 6.6**, the current thresholds of 14 and 10 were chosen because they give a meaningful ambiguity rate while not being so aggressive that most inputs are flagged. Applications that require very confident outputs could raise these thresholds. Applications that need to flag any uncertain input could lower them.

6.12 Performance

Training takes approximately 3 to 4 hours on CPU for the full 43,410-sample dataset. The bottleneck is the topology computation step, which runs for every training sample. With GPU-accelerated embedding encoding, the embedding step is reduced from about 10 minutes to about 2 minutes, but the topology computation is CPU-bound.

Inference takes approximately 2 to 4 seconds per query on a standard laptop CPU. This is acceptable for interactive use. The topology computation (finding neighbours and computing persistence) is the bottleneck at inference time as well.

6.13 Limitations

The known limitations of the current system are:

- **Social Ambiguity Blind Spots:** The system cannot detect social ambiguity that is not geometrically visible in the embedding space. “I am fine” is a well-known socially ambiguous phrase, but it does not appear ambiguous in the geometric sense because training data biases its neighbourhood toward clearly Positive samples.
- **Slang and Informal Language:** The system struggles with slang, abbreviations, and informal language that is not well-represented in the training data. Words like “yolo”, “fomo”, “lol”, and “idk” carry emotional meaning but the embedding model may not place them accurately because such terms are underrepresented in the model’s training data. As a result, unusual or newly emerging slang expressions may lead to incorrect or ambiguous predictions. This highlights a limitation of data-driven models: understanding of language is restricted to patterns seen during training. Handling such language would require additional preprocessing using slang dictionaries or domain-specific training data.

- **Threshold Calibration:** The ambiguity thresholds were set manually based on observation and experimentation. There is no labelled dataset of "ambiguous social media posts" that could be used for proper threshold calibration.
- **Language Constraint:** The system works only on English text. It was trained on Reddit comments and may not generalize well to other social media platforms with different writing styles.
- **Inference Bottleneck:** Inference is slower than a standard classifier because of the live topology computation. For large-scale deployment, this would need to be optimized.

Chapter 7

Conclusion and Future Work

7.1 Summary

This project built an emotion detection system for social media text that does something most existing systems do not: it knows when not to answer. Instead of always returning a confident label, the system can output Ambiguous when the input text is genuinely unclear or sits in a boundary region between emotion classes.

The system combines three types of features. Semantic embeddings from the `all-mpnet-base-v2` model capture the meaning of the text. TF-IDF features capture the specific words and phrases present. Topological features from Vietoris-Rips persistent homology capture the geometric structure of the neighbourhood around the input in the embedding space.

Three XGBoost classifiers trained in a one-vs-rest configuration produce probabilities for the Positive, Negative, and Neutral classes. On top of this, an ambiguity detection layer combines two signals: the number of H1 loops in the local persistence diagram and the confidence gap between the top two class probabilities. Three decision rules convert these signals into a final output, which can be Positive, Negative, Neutral, or Ambiguous.

7.2 What Was Achieved

The key achievements of this project are:

A working emotion classifier was built on the full GoEmotions dataset (43,410 training samples, 5,427 test samples), achieving an accuracy of 0.59 and a weighted F1 score of 0.60. These numbers reflect the genuine difficulty of the dataset, not a fundamental failure of the approach.

A topological ambiguity detection mechanism was introduced. This mechanism uses local Vietoris-Rips persistent homology computed fresh at inference time for every input. The number of H1 loops in the persistence diagram provides a geometric signal of ambiguity that is independent of the classifier's output.

The system correctly identifies genuinely ambiguous sentences. "i dont know how i feel" returns AMBIGUOUS due to an extreme loop count of 15. "i will kill you" returns AMBIGUOUS due to a very low confidence gap of 0.06. "do this" returns AMBIGUOUS due to moderate topology combined with low classifier confidence.

The system provides interpretable outputs. When it returns AMBIGUOUS, it explains which rule triggered the decision. This is important for practical deployment where users need to

understand and trust the system's decisions.

7.3 Advantages of the Approach

The main advantage of using topology is that it provides a signal that is independent of the classifier. A classifier can be trained to be confident on ambiguous inputs if it has seen many similar examples. Topology looks at the structure of the data itself.

The approach is also principled. The connection between geometric complexity (H1 loops) and semantic ambiguity (text near the boundary of emotion clusters) is not arbitrary. It reflects a real property of the embedding space: texts that are semantically ambiguous are found in geometrically complicated regions.

The system is also explainable. Every output comes with a reason. This is a property that most deep learning systems do not have.

7.4 Limitations

The system has the following known limitations:

- **Threshold Calibration:** The thresholds for ambiguity detection (14, 10, 0.30, 0.10) were set manually based on experimentation. Without a labelled ambiguity dataset, these cannot be calibrated systematically.
- **Social Ambiguity Blind Spots:** The system cannot detect social ambiguity that is not visible in the embedding space. "I am fine" is socially complex but does not appear geometrically ambiguous because the training data biases its neighbourhood toward clearly Positive samples.
- **Slang and Informal Language:** The system struggles with slang and informal abbreviations. Words like "yolo", "fomo", "lol", and "idk" carry emotional meaning but may not be well-represented in the training data. The embedding model may not place such terms accurately, leading to incorrect or ambiguous predictions.
- **Inference Bottleneck:** Inference is slower than a standard classifier because of the live topology computation. For large-scale systems, this would need to be optimized.
- **Language Constraint:** The system works only on English text.

7.5 Future Work

There are several directions where this work can be extended:

- **Labelled Ambiguity Dataset:** The most important improvement would be creating a labelled ambiguity dataset. If a set of social media posts were annotated by humans for whether they are genuinely ambiguous, the ambiguity thresholds could be calibrated properly and the system's performance on ambiguity detection could be measured numerically.

- **Faster Topology Computation:** A faster topology computation method would make the system more practical for deployment. One option is to precompute topological signatures for regions of the embedding space and do a lookup at inference time instead of computing fresh persistence for every input.
- **Fine-Grained Emotion Classes:** Extending the system to handle fine-grained emotion classes, rather than the collapsed three-class version, would be a more challenging but more useful task. The GoEmotions dataset supports 28 classes, and a system that can detect ambiguity at this level would be much more informative.
- **Sarcasm and Irony Detection:** Adding a dedicated sarcasm and irony detection module would address many of the cases where social ambiguity is not captured by the topological signal. Sarcasm in particular is common in Reddit comments and is a major source of error in sentiment analysis systems.
- **Multilingual Support:** Supporting multiple languages through multilingual embeddings would make the system applicable to a much wider range of social media content.
- **User Interface:** Finally, a web interface would make the system accessible to non-technical users. The current command-line interface is sufficient for development and testing but not for general use.

7.6 Final Thoughts

Emotion detection is a genuinely hard problem. Human language is full of nuance, context-dependence, and social complexity. A system that always gives a confident answer is not always being honest.

This project shows that it is possible to build a system that knows what it does not know and says so clearly. The combination of geometric structure from topology and probabilistic output from a classifier is a promising direction for building more reliable and trustworthy natural language processing systems. The GoEmotions dataset, with all its noise and imbalance, provides a realistic and challenging test bed for exploring this direction.

The work here is a starting point. There is much more to be done in characterizing ambiguity in text, calibrating geometric signals, and connecting mathematical structures from topology to the practical needs of emotion understanding.

Bibliography

[1] D. Demszky, D. Movshovitz-Attias, J. Ko, A. Cowen, G. Nemade, and S. Ravi, *GoEmotions: A Dataset of Fine-Grained Emotions*, Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 2020.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, arXiv preprint, 2018.

[3] H. Edelsbrunner, D. Letscher, and A. Zomorodian, *Topological Persistence and Simplification*, Discrete and Computational Geometry, 2002.

[4] G. Tauzin, U. Lupo, L. Tunstall, J. B. Rose, M. Caorsi, A. Medina-Mardones, A. Becca, and C. Otani, *Giotto-TDA: A Topological Data Analysis Toolkit for Machine Learning and Data Exploration*, Journal of Machine Learning Research, 2021.

[5] A. V. Smith et al., *Topological Quantification of Ambiguity in Semantic Search*, arXiv preprint, 2019.

[6] *An Experimental Analysis of Data Annotation Methodologies for Emotion Detection in Short Text*, MDPI, 2023.

[7] S. Canavan, *AffectiveTDA: Using Topological Data Analysis to Improve Analysis and Explainability in Affective Computing*.

[8] *Emotion Classification Using BERT: A Comprehensive Study*, Journal of Propulsion Technology, 2024.

[9] *Blowfish: Topological and Statistical Signatures for Quantifying Ambiguity in Semantic Search*, arXiv, 2021.

[10] *Topological Data Analysis and Topological Deep Learning Beyond Persistent Homology: A Review*, PMC, 2023.

[11] *TDAvec: Computing Vector Summaries of Persistence Diagrams for Topological Data Analysis*, arXiv, 2022.

[12] *Large Language Models on Fine-Grained Emotion Detection Dataset*, arXiv, 2023.

[13] *Fine-Grained Emotion Detection on GoEmotions*, arXiv, 2021.

[14] *Emotion Topology: Extracting Fundamental Components of Emotions from Text Using Word Embeddings*, PMC, 2021.

[15] *AI Meets Psychology: Large Language Models in Psychotherapy Contexts*, Taylor & Francis, 2023.

[16] *Deep Learning for Early Detection of Anxiety and Depression Using Twitter Data*, 2022.

[17] *ModernBERT for GoEmotions Dataset*, 2024.

[18] *Topological Data Analysis for EEG Processing*, PMC, 2021.

[19] *Topological Data Analysis in Cardiovascular Signals*, PMC, 2022.

[20] *Sentiment Analysis: Why Models Miss Sarcasm*, 2023.

[21] *Persistence Kernels for Classification*, 2019.

[22] *Teaspoon: A Python Package for Topological Signal Processing*, Journal of Open Source Software, 2020.

[23] *The Persistent Laplacian for Data Science*, Proceedings of Machine Learning Research, 2020.

[24] *Simplicial SMOTE: Oversampling Solution for Imbalanced Learning*, 2021.

[25] *TS-SMOTE: Improved SMOTE Method*, MDPI, 2022.

[26] *MARCH: Evaluating Ambiguity Interpretation and Multi-hop Inference*, arXiv, 2023.

[27] *Emotion Classification with XGBoost Using EEG Data*, 2020.

[28] *Information Theory of Persistent Homology*, MDPI, 2022.

[29] P. Michel et al., *Does the Geometry of Word Embeddings Help Document Classification?*, 2017.

[30] N. Reimers and I. Gurevych, *Sentence-BERT: Sentence Embeddings Using Siamese BERT Networks*, EMNLP, 2019.

[31] X. Zhu, *Persistent Homology for NLP*, IJCAI, 2013.